G H RAISONI INTERNATIONAL SKILL TECH UNIVERSITY, PUNE

DEPARTMENT OF COMPUTER APPLICATION

Bachelor of Business Administration in Computer Applications (BBA CA)

COURSE ASSIGNMENT PORTFOLIO

Project: College AI Chatbot

Comprehensive Solutions for Assignments 1, 2, 3 & 4 • Semester III

Submitted By : Kanishk Singh (Roll No: ISTU25010120001)
Vibhor Saini (Roll No: ISTU25010120002)

Class & Semester : SY BBA CA • Semester III

Subject : Software Engineering & Project Work

Project Guide : Asst. Prof. Sheela Satav

Institution : G H Raisoni International Skill Tech University, Pune

Academic Year 2026-27

TABLE OF CONTENTS

COMPLETE INDEX OF COURSEWORK ASSIGNMENTS • SEMESTER III

Asgn No.	Assignment Title & Covered Topics	Questions	Page No.
Assignment 1	**Project Overview, Requirement Analysis & System Architecture** Project description, advantages, elicitation methodology, tech stack, data flow, 3NF entities, drawbacks, testing & future roadmap.	Questions 1 – 14	**3 – 5**
Assignment 2	**Unified Modeling Language (UML) & System Design Diagrams** UML definition, structural vs behavioral taxonomy, and high-resolution diagrams: Use Case, Class, Sequence, Activity & Component diagrams.	Questions 1 – 4 (5 Full Diagrams)	**6 – 11**
Assignment 3	**Software Project Life Cycle & SDLC Models** Project Life Cycle definition & characteristics; detailed analysis of Waterfall, Iterative (Project Methodology), Spiral, V-Shaped, and Agile models.	Questions 1 – 2 (5 SDLC Models)	**12 – 13**
Assignment 4	**Software Testing, Quality Assurance, Metrics & SCM** Testing definitions, risk identification, SCM, debugging, configuration audit, black box vs white box table, metrics, version control, OOSE, testing types, and automated vs manual testing table.	Questions 1 – 11 (2 Comparison Tables)	**14 – 16**

SUBMISSION CERTIFICATION :

This portfolio compiles the complete, authentic academic solutions for Assignments 1 through 4 for the *College AI Chatbot* project. All diagrams, empirical analyses, architectural breakdowns, and testing matrices reflect the live production system deployed at G H Raisoni International Skill Tech University, Pune.

Student Signature
Kanishk Singh & Vibhor Saini

Asst. Prof. Sheela Satav
Project Guide • Dept. of Computer Application

ASSIGNMENT 1

TOPIC: PROJECT OVERVIEW, REQUIREMENTS & IMPLEMENTATION ARCHITECTURE

1. Describe your project.

Project Title: *College AI Chatbot* is an automated, intelligent conversational assistant and campus self-service knowledge portal designed and developed specifically for **G H Raisoni International Skill Tech University, Pune**. The primary objective is to provide students, parents, and administrative staff with instant, round-the-clock (24/7) automated responses to frequently asked institutional queries spanning admissions, fee structure, semester examination timetables, 4-tier attendance detention rules, syllabus/course details, library timings, faculty contact directories, hostel/mess regulations, campus placement track records, and location transit directions for both Wagholi and Yerawada Pune campuses.

The system employs a hybrid natural language processing engine combining exact pattern matching with typo-tolerant fuzzy matching (via Python's difflib.SequenceMatcher) over an indexed relational database. It incorporates authenticated student access where enrolled students log in via their institutional registration numbers (e.g., ISTU25010120001) to fetch real-time personalized records including individual attendance percentage, 4-tier detention risk tier, pending fee balance, cumulative CGPA, assigned faculty mentor, and hall ticket eligibility. Furthermore, the application features bi-directional voice accessibility via Web Speech API (Speech-to-Text and Text-to-Speech audio readout), Progressive Web App (PWA) offline-ready installability, and an administrative analytics command center with live KPI metric cards and CSV audit logging.

2. Advantages of your project.

- **24/7 Instant Self-Service:** Eliminates long student queues at administrative and departmental enquiry counters by offering sub-second automated answers outside office hours and during exam seasons.
- **Typo-Tolerant Conversational Robustness:** Accommodates student spelling errors, typos, and phonetic variants (e.g., “hostal fee”, “atendance rule”) using dual-tier fuzzy matching without throwing jarring syntax errors.
- **Personalized Academic & Detention Insights:** Automatically computes student attendance ratios and classifies them into institutional standing tiers (Good Standing ≥ 75%, Warning 60–74.9%, Critical 50–59.9%, Course Detention < 50%), providing actionable remediation guidance.
- **Multi-Modal Voice Accessibility:** Allows hands-free operation through microphone Speech-to-Text (STT) and synthesized spoken voice output (TTS) powered by browser Web Speech API and Google Text-to-Speech (gTTS).
- **Administrative Control & CSV Audit Exports:** Enables non-technical staff to update FAQs dynamically, review satisfaction feedback ratings (👍/👎), and export query logs to CSV for compliance audits.
- **PWA Native Experience with Zero Licensing Overhead:** Operates as an installable app across smartphones, tablets, and desktops without third-party app store commissions, built entirely on an open-source technology stack.

3. How do you gather the requirements of your project?

Requirements engineering followed an empirical, multi-stage stakeholder elicitation process:

a) Enquiry Counter Observation & Student Surveys: Documented repetitive inquiries asked by students and visitors at GHRISTU administrative desks over past semesters. These high-frequency inquiries were organized into 14 distinct categories (Admissions, Fees, Timetable, Attendance, Library, Courses, Faculty, Branch Change, Placements, Hostel, Scholarships, Clubs, Helplines, Campus Transit).

b) Institutional Policy Review: Examined official university guidelines regarding course registration, 75% mandatory attendance policies, examination hall ticket eligibility, and administrative escalation workflows.

c) Mentor Guidance & Prototype Reviews: Consulted project guide **Asst. Prof. Sheela Satav** to identify non-functional requirements including responsive theme switching (Light/Dark mode), cryptographic password hashing (PBKDF2/scrypt), database normalization (3NF), and automated unit test verification.

4. How and where do you implement your project?

Implementation Stack: Developed using Python 3 (Flask RESTful framework) for backend API routing, SQLite3 for relational persistence, and HTML5, modern CSS3 (Flexbox/CSS Grid), and asynchronous vanilla JavaScript (ES6+ fetch() API) for frontend user interface rendering.

Deployment Environment: Source code is version-controlled via Git and hosted in a public GitHub repository (kanishksingh-01/college-ai-chatbot). It is deployed live on **Render Cloud Platform** using continuous deployment (CI/CD) pipelines, accessible globally at: https://college-ai-chatbot-c94w.onrender.com. Every code push to the main branch automatically triggers automated builds, dependency resolution via requirements.txt, and zero-downtime service redeployment.

5. Which software you used in your project?

- **Visual Studio Code (VS Code):** Primary Integrated Development Environment (IDE) used for code authoring, linting, debugging, and terminal execution.
- **Python 3.10+:** High-level programming language executing the server runtime, natural language processing, and database transaction logic.
- **Flask (v3.0.3):** Lightweight, WSGI web application framework used to construct RESTful endpoints (/chat, /login, /admin, /export).
- **SQLite 3:** Serverless, zero-configuration relational database engine embedded directly into the application for storing institutional FAQs, student profiles, and chat logs.
- **Werkzeug (v3.0.3):** Cryptographic security module handling secure password hashing using salt-extended PBKDF2 with SHA-256 and scrypt algorithms.
- **Web Speech API:** Native browser APIs (webkitSpeechRecognition and SpeechSynthesis) for real-time speech-to-text and text-to-speech audio rendering.
- **Git & GitHub:** Distributed version control system and cloud repository hosting for collaborative commit tracking and continuous delivery.
- **Render Cloud Hosting:** Fully managed cloud application hosting platform running Linux container environments for public internet availability.
- **Google Chrome & Safari:** Web browsers utilized for cross-platform debugging, DevTools performance auditing, responsive testing, and PWA manifest validation.

6. What methodology is used for the project?

The project was executed using the **Iterative and Incremental Software Development Life Cycle (SDLC) Model**. Rather than attempting a rigid one-pass deployment, the chatbot was architected across 6 progressive iteration cycles:

• *Iteration 1 (Core Prototype):* Console-based keyword matching algorithm and baseline response dictionary.
• *Iteration 2 (Persistence & Web Interface):* SQLite database schema creation (3NF) and responsive HTML/CSS/JS chat screen.
• *Iteration 3 (Student Authentication):* Student portal login enabling personalized queries for attendance percentage, fees, CGPA, and mentor.
• *Iteration 4 (Administrative Governance):* Password-protected admin command center with live KPI metric cards and CSV audit logging.
• *Iteration 5 (Accessibility & PWA):* Web Speech API voice interaction (STT/TTS), Dark/Light mode theme toggle, and PWA Service Worker caching.
• *Iteration 6 (Testing & Cloud Deployment):* Automated unit testing suite (test_suite.py), Render CI/CD cloud deployment, and system verification.

Each iteration underwent thorough functional testing and verification before incorporating subsequent modules, ensuring high system stability and zero regression flaws.

7. What is the hardware or software of the project?

Hardware Requirements:

• *Client Device:* Any standard PC, laptop, tablet, or smartphone equipped with at least 1 GB RAM, internet connectivity, microphone (for voice query input), and audio speakers/headphones (for spoken feedback).
• *Cloud Server:* Render Linux Cloud Container provisioned with 512 MB RAM, 0.1 vCPU (shared), and persistent SSD block storage.

Software Requirements:

• *Client-Side:* Modern web browser supporting HTML5, CSS3, ES6 JavaScript, and Web Speech API (Google Chrome 90+, Safari 14.1+, Microsoft Edge 90+, Mozilla Firefox 88+).
• *Server-Side:* Linux OS, Python 3.10 runtime environment, Flask WSGI framework, SQLite3 database, and Gunicorn web server.

8. What is your role in the project?

The project was developed collaboratively by a dedicated two-member academic team:

• **Kanishk Singh (Roll No: ISTU25010120001):** Lead Backend Developer & System Architect — Responsible for designing the Flask application structure, NLP query matching algorithms, fuzzy similarity logic (difflib), normalized 3NF database schema, student personalized data retrieval, automated test suite engineering (test_suite.py), Git version control workflow, and Render cloud deployment pipeline.
• **Vibhor Saini (Roll No: ISTU25010120002):** Frontend & UI/UX Specialist — Responsible for responsive client-side interface development, Light/Dark theme CSS engineering, Web Speech API integration (voice recognition and text-to-speech audio chimes), admin dashboard UI design, and manual cross-device testing.

9. Explain the data flow of your application.

The data flow follows a synchronous client-server request-response pattern:

1. **Input Capture:** The user enters a question via keyboard typing, selects a quick-reply category button, or speaks through the microphone (which converts speech to text via webkitSpeechRecognition).
2. **HTTP Request:** JavaScript packages the input into a JSON payload { message: "...", user_id: "..." } and sends an asynchronous HTTP POST request to the Flask /chat endpoint.
3. **Normalization & Intent Analysis:** The backend strips special characters, normalizes tokens to lowercase, and checks if the student is authenticated and requesting personal data (e.g., “my attendance”). If so, it queries the students table directly.
4. **Fuzzy Keyword Matching:** For general institutional FAQs, match_query() computes token overlap scores and similarity ratios against the college_info table to determine the optimal response.
5. **History Logging:** The user query, matched response, timestamp, and active session identifiers are committed to the chat_history table.
6. **Client Rendering:** The JSON response returns to the browser. JavaScript dynamically updates the DOM, renders tabular responses as structured HTML tables, and triggers the Web Speech synthesis engine to speak the answer aloud.

10. What are the entities in your project?

The system models four core database entities normalized in Third Normal Form (3NF):

- **Student Entity (students table):** Models enrolled student records. Attributes: roll_no (Primary Key, e.g., 'ISTU25010120001'), name, attendance (percentage real), fees_due (numeric balance), exam_date, course (e.g., 'BBA CA'), semester ('Semester III'), cgpa, and mentor (assigned faculty mentor).
- **CollegeInfo Entity (college_info table):** Represents the centralized institutional knowledge base. Attributes: id (Primary Key, autoincrement), category (one of 14 domains, e.g., 'Admissions', 'Fees'), keywords (comma-separated indexing tokens), and answer (formatted response text).
- **ChatHistory Entity (chat_history table):** Tracks conversational audit trails. Attributes: id (Primary Key), roll_no (Foreign Key referencing students), user_query, bot_response, timestamp, and feedback (satisfaction rating: thumbs up / thumbs down).
- **Admin Entity (admin table):** Manages administrative access control. Attributes: username (Primary Key), password_hash (salted PBKDF2/scrypt cryptographic hash), role, and session_state.

11. What is the drawback of your system?

- **Closed-Domain Rule Knowledge:** Because it operates on deterministic keyword and fuzzy matching rather than unconstrained large language models, the chatbot cannot synthesize novel answers for topics not pre-configured in the college_info database.
- **Administrative Maintenance Dependency:** Institutional accuracy relies upon administrative staff keeping semester dates, fee circulars, and faculty contact lists updated inside the admin portal.
- **Free Cloud Tier Cold-Start Latency:** When hosted on Render's free tier, server instances spin down during periods of 15-minute inactivity, requiring approximately 30 to 40 seconds to “wake up” on the initial cold request.
- **SQLite Concurrency Limitations:** SQLite employs file-level database locking during write transactions, which is sufficient for college-scale traffic but would require migration to PostgreSQL for massive enterprise loads.

12. How do you test your application?

Testing was conducted through a combination of automated and manual verification strategies:

• **Automated Unit Testing:** Implemented a test suite using Python's standard unittest framework (test_suite.py). Automated tests verify keyword scoring heuristics, fuzzy matching thresholds (confirming “hostal fee” resolves accurately to the Hostel category), personalized student queries (attendance, CGPA, mentor lookup), and HTTP response status codes with a 100% pass rate.

• **Manual Functional & UI Testing:** Conducted cross-device manual testing on desktop and mobile browsers (Chrome, Safari, Edge). Verified voice input transcription accuracy in noisy ambient conditions, dark mode contrast compliance, student authentication failure handling, administrative FAQ CRUD operations, and CSV export integrity.

• **Regression Testing:** Executed end-to-end user journeys on the live Render cloud deployment after every Git commit to confirm zero broken features.

13. From where do you get reference, guide for your project work?

Academic Guidance: Conceptual architecture, software engineering methodologies, UML modeling rules, and syllabus compliance were guided under the direct supervision of project guide **Asst. Prof. Sheela Satav**, Department of Computer Application, G H Raisoni International Skill Tech University, Pune.

Technical Documentation & Reference Sources:

• Official Flask Documentation (Pallets Projects) for WSGI routing, application context, and session management.

• Python Standard Library Documentation for difflib.SequenceMatcher, sqlite3, and unittest modules.

• MDN Web Docs (Mozilla Developer Network) for Web Speech API specifications (SpeechRecognition, SpeechSynthesis) and PWA Service Worker caching strategies.

• Software Engineering Literature: *Software Engineering: A Practitioner's Approach* by Roger S. Pressman; *UML Distilled* by Martin Fowler.

14. In the future, what changes will you make in your application?

- **Generative AI & RAG Architecture:** Integrate Google Gemini API / OpenAI API with Retrieval-Augmented Generation (RAG) over official college PDF syllabi and prospectus documents for contextual reasoning.
- **Multi-Channel WhatsApp & Telegram Bot:** Integrate official WhatsApp Business API to allow students to query the chatbot directly from their messaging apps without opening a browser.
- **Multi-Lingual Regional Support:** Add native voice and conversational support for Marathi and Hindi alongside English to support regional diversity.
- **ERP & RFID Live Synchronization:** Establish live webhook integration with the university's central ERP database for real-time lecture attendance biometric sync and automated online fee payment receipt generation.

ASSIGNMENT 2

TOPIC: UNIFIED MODELING LANGUAGE (UML) • THEORY, TAXONOMY & APPLICATION

1. What is UML?

Unified Modeling Language (UML) is an industry-standard, standardized visual modeling language developed by Grady Booch, Ivar Jacobson, and James Rumbaugh ("The Three Amigos") and maintained by the Object Management Group (OMG). It provides software engineers, systems analysts, and architects with a standardized graphical notation to **specify, visualize, construct, and document** the artifacts of an object-oriented software system. UML is programming-language independent; it models real-world business domains, system static structures, behavioral dynamics, and physical deployment topologies before and during software development.

2. What is the use of UML?

In modern software engineering, UML serves several vital engineering purposes:

• **Architectural Blueprinting:** Provides an unambiguous visual blueprint of system components, class hierarchies, and database schemas before coding commences, mitigating costly architectural re-engineering.

• **Cross-Functional Communication:** Bridges technical developers, database administrators, academic project guides, and non-technical stakeholders through a universal graphical medium.

• **Behavioral & Workflow Modeling:** Illustrates the exact sequence of message passing, state transitions, and conditional decision branching between interacting runtime objects.

• **Formal System Documentation:** Acts as permanent, version-controlled technical documentation that accelerates project onboarding, refactoring, and future feature maintenance.

• *Application in College AI Chatbot:* UML was instrumental in mapping user roles (Student and Admin actors), structural associations between entities (Student to ChatHistory), and the asynchronous query-response lifeline execution.

3. Explain types of UML diagram with example.

UML (version 2.5) defines 14 official diagram types divided into two overarching categories:

A. Structural Diagrams (Static System Architecture):

1. *Class Diagram:* Illustrates the system's static structure by showing classes, attributes, operations (methods), and associations. Example: In our project, the Student class with attributes (roll_no, attendance, cgpa) associated in a 1-to-0..* relationship with ChatHistory.
2. *Object Diagram:* Represents a snapshot of runtime instances with assigned slot values at a specific moment in time. Example: An object instance kanishk : Student with roll_no = "ISTU25010120001" and attendance = 82.5%.
3. *Component Diagram:* Depicts how software modules, libraries, and executables are wired together via interfaces. Example: ChatbotUI component connecting via REST API to FlaskServer and SQLite3 DB.
4. *Deployment Diagram:* Visualizes the hardware execution topology and runtime artifacts. Example: User client browser hardware executing over HTTPS to Render Linux cloud servers.

B. Behavioral Diagrams (Dynamic System Execution & Interaction):

1. *Use Case Diagram:* Illustrates external actors and their observable interactions with system use cases within a defined system boundary. Example: Student executing "Ask College FAQs" and "Student Login".
2. *Sequence Diagram:* Models time-ordered message exchange between participating object lifelines. Example: Student enters voice query → UI sends POST → Flask queries SQLite → Audio synthesis returns to student.
3. *Activity Diagram:* Represents procedural workflow, control flow branching (decision diamonds), and concurrent activity forks. Example: The decision flow checking whether a user query is authenticated before executing attendance detention checks.
4. *State Machine Diagram:* Models the state transitions of an entity across its lifecycle in response to external events. Example: A query transitioning from Received → Parsing → Matched → Logged.

4. Draw UML Diagrams of your project — (a) Use Case Diagram

System Boundary & Actor Interactions: Models the functional requirements of the College AI Chatbot. The system includes two primary actors: **Student** (queries FAQs, uses voice input/TTS, logs in for attendance/CGPA, submits feedback ratings, exports chat transcripts) and **Admin** (authenticates with hashed password, views KPI analytics, updates FAQ database, and exports CSV audit logs). It incorporates the mandatory <<include>> relationship (Ask College FAQs includes querying campus address and fees) and the <<extend / exclude>> dependency (Voice Input extends the standard text query use case).

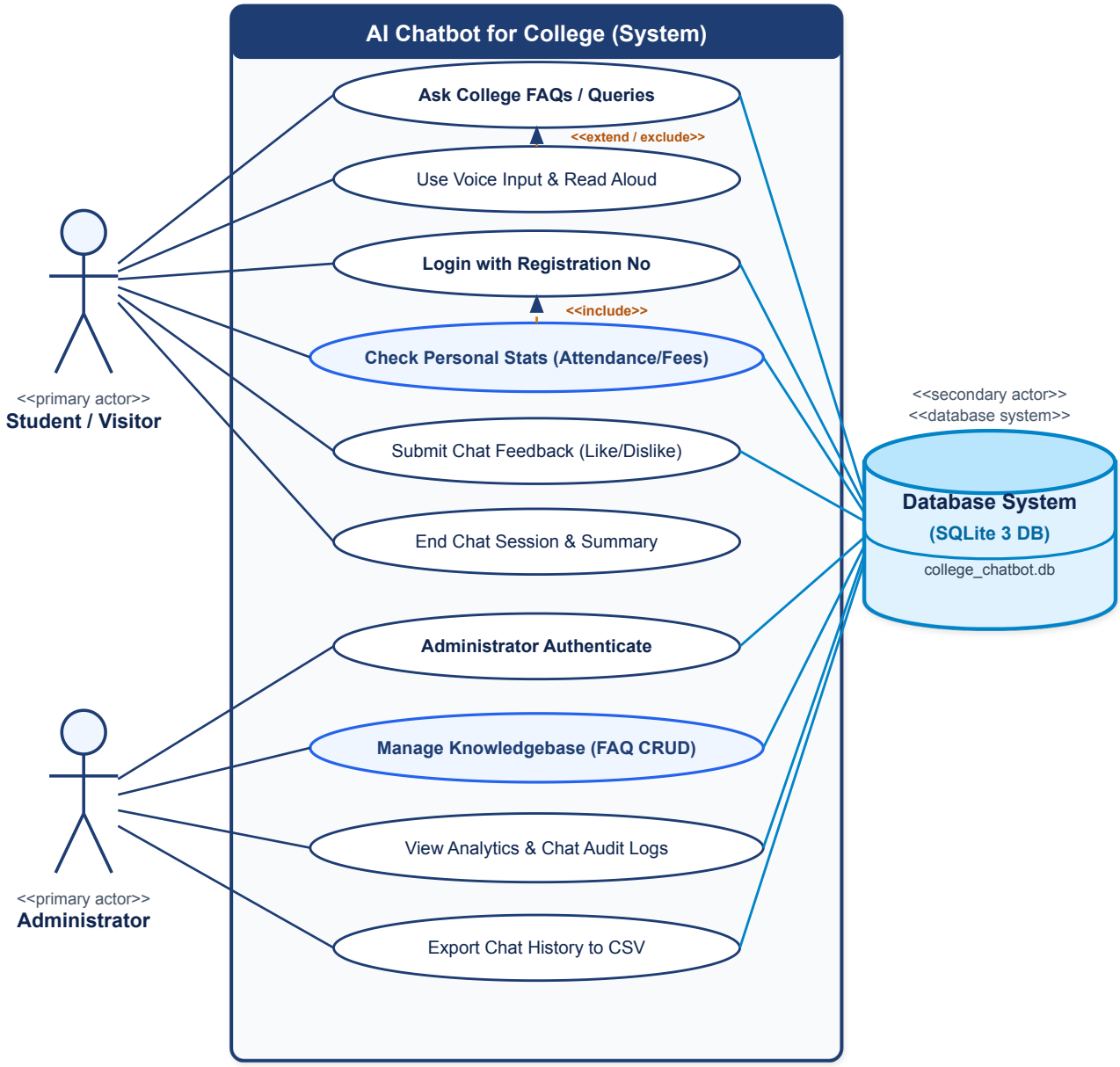


Figure 2.1: Complete Use Case Diagram with System Boundary, Database Cylinder, and Include/Extend Dependencies.

4. Draw UML Diagrams of your project — (b) Class Diagram

Object-Oriented Structure: Depicts the static class structure of the application. An abstract base class User provides common authentication credentials (username, password_hash, role) inherited via generalization (hollow triangle) by both Student and Admin. Structural associations specify exact multiplicities (1 Student submits 0..* ChatHistory logs; 1 Admin manages 0..* CollegeInfo FAQ entries; CollegeInfo is matched 0..1 in 0..* ChatHistory records).

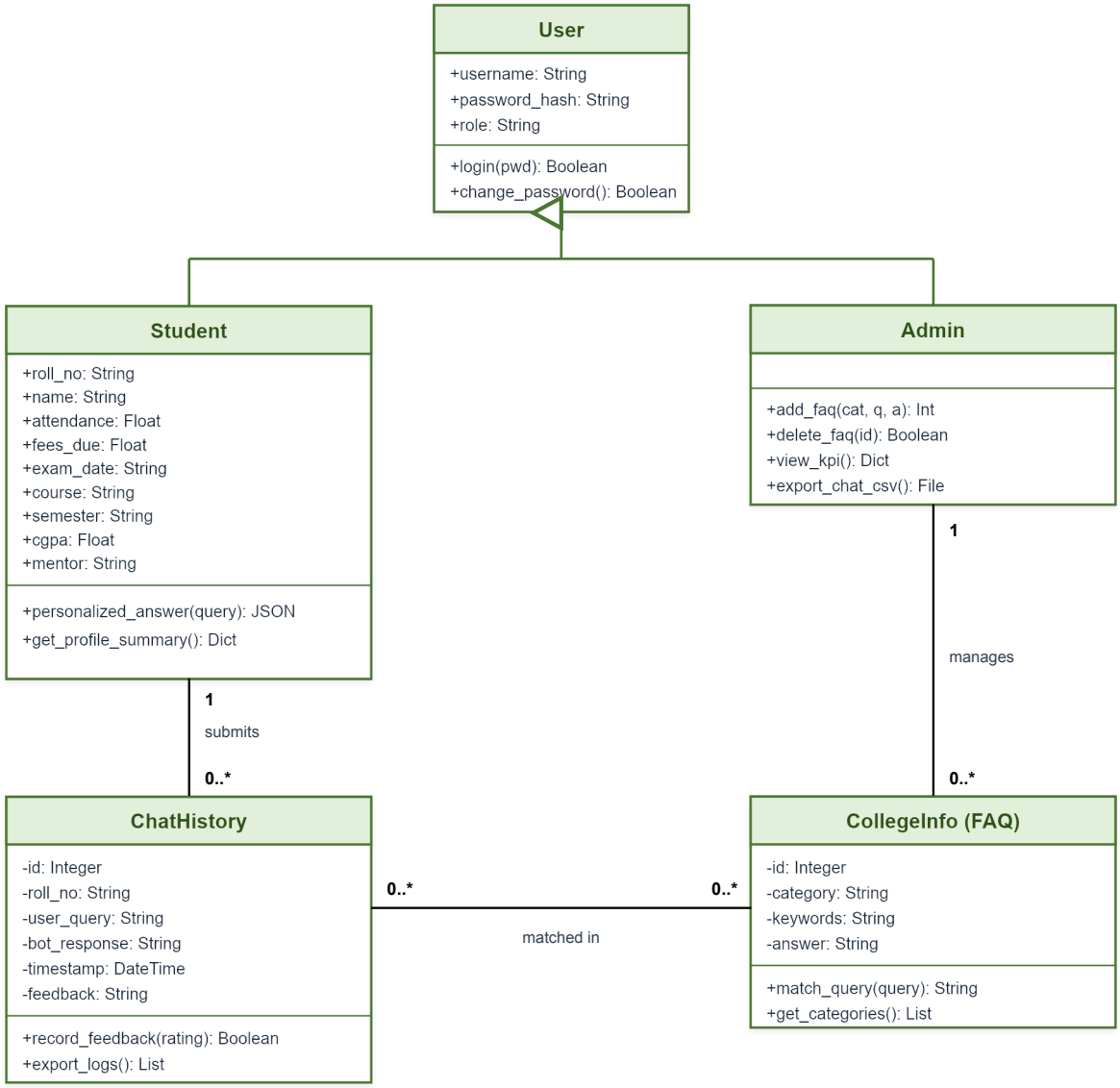


Figure 2.2: Standard Class Diagram with Attributes, Visibility Symbols (+/-), Methods, Inheritance & Multiplicities.

4. Draw UML Diagrams of your project — (c) Sequence Diagram

Temporal Message Sequencing: Illustrates the chronologically ordered exchange of messages during a student session. The sequence depicts lifelines for Student, Chatbot UI (Browser), Flask Server (app.py), NLP & Fuzzy Matcher, and SQLite Database (chatbot.db). It models synchronous request invocations (solid lines with arrows), database commits, asynchronous reply returns (dashed lines with open arrowheads), and feedback rating logging.

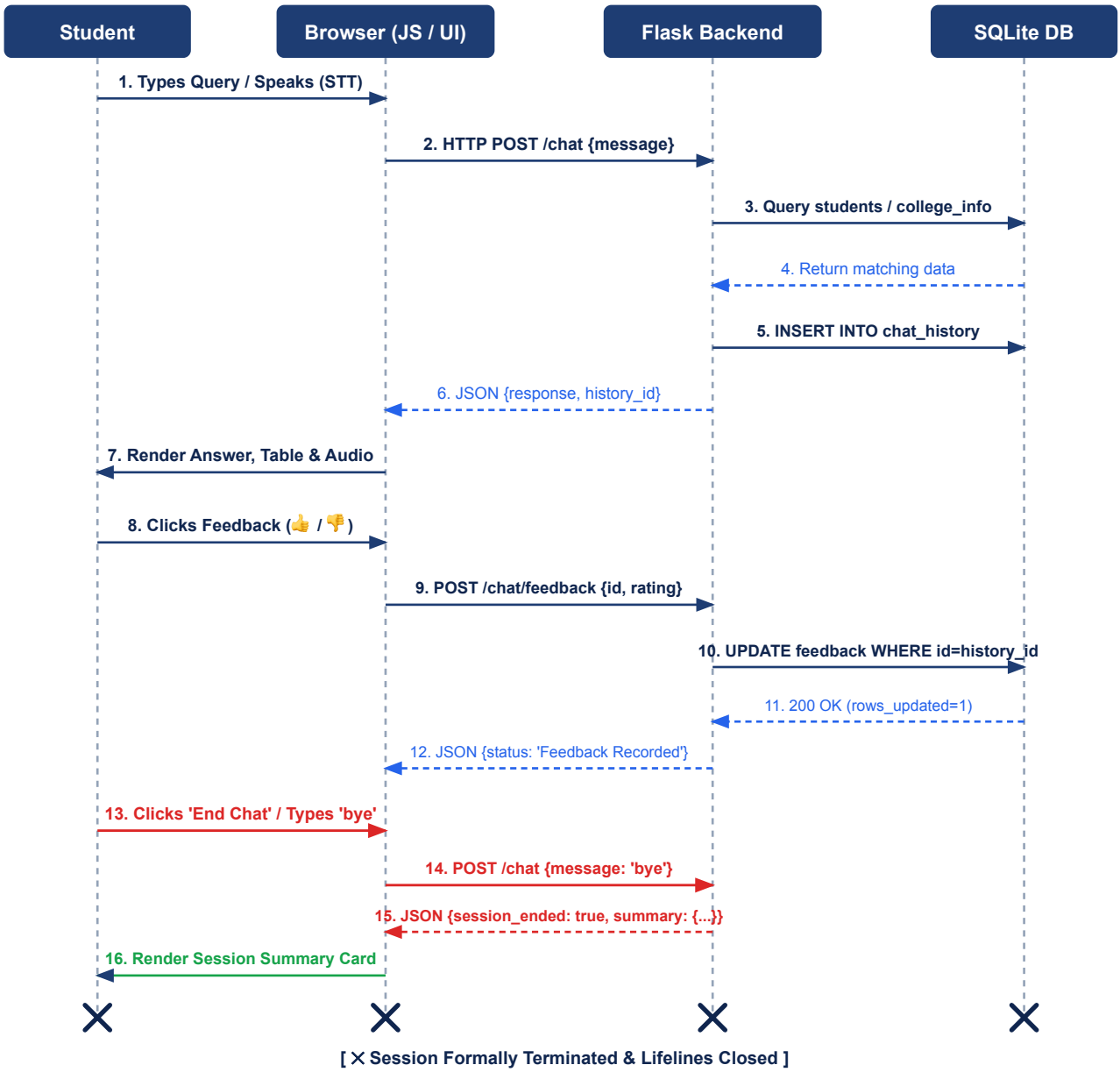


Figure 2.3: Sequence Diagram Modeling Message Flow, Activation Bars, Database Queries, and Audio Feedback.

4. Draw UML Diagrams of your project — (d) Activity Diagram

Workflow & Decision Logic: Traces the end-to-end execution flow of a user interaction. Starting from the initial node (filled circle), the activity captures user input, forks for optional Web Speech Speech-to-Text transcription, evaluates decision diamonds for student authentication status, checks 4-tier detention boundaries, runs fuzzy keyword algorithms, logs interaction records to SQLite, synthesizes spoken TTS audio, and terminates at the final state.

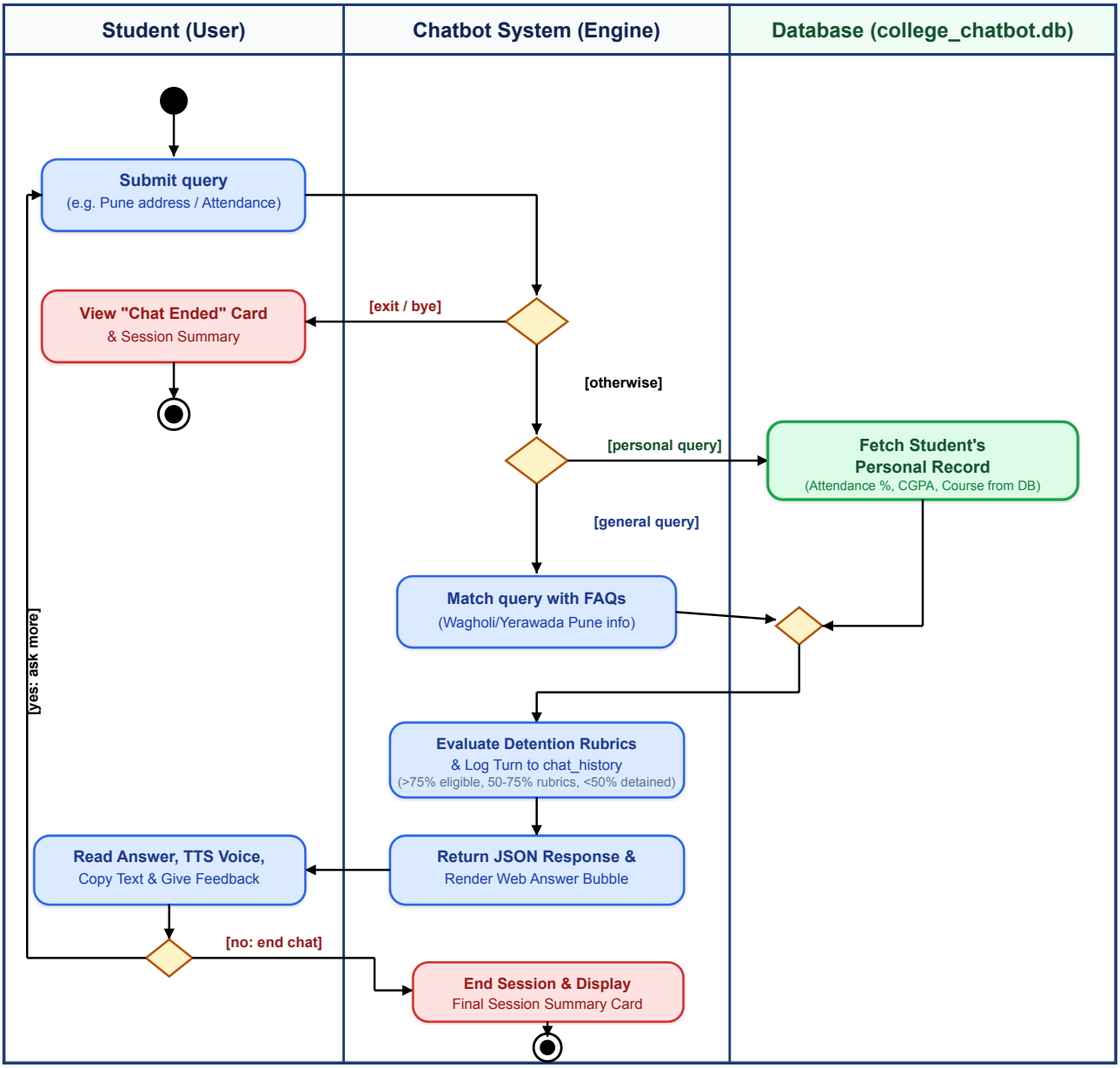
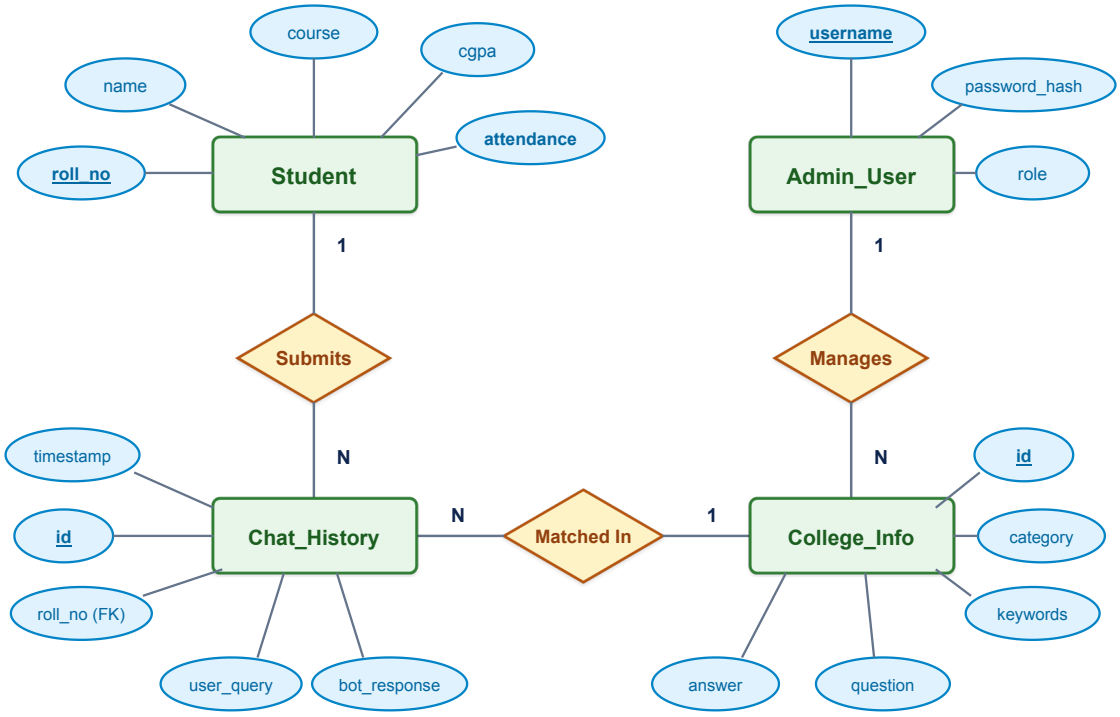


Figure 2.4: Activity Diagram Tracing Branching Logic, Intent Resolution, Detention Categorization & Termination.

4. Draw UML Diagrams of your project — (e) Component Diagram

Modular Component Architecture: Visualizes the structural relationships between autonomous software modules. The architecture illustrates three principal tiers: Client Presentation Tier (ChatbotUI), Application Controller Tier (FlaskServer hosting NLP_Matcher and Personalization Engine), and Data Persistence Tier (SQLite3 DB). External dependencies (Web Speech API and Google Text-to-Speech Engine) bind through standardized interface sockets.

ER Diagram (Classic Notation): AI Chatbot for College



Schema & Project Knowledge Base Defense:

- **Student Entity:** Stores attendance %, CGPA & evaluates 4-tier detention policy.
- **College_Info Entity:** Contains Pune campuses (Wagholi & Yerawada) & exam rules.
- **Chat_History Entity:** Logs each turn with student roll_no FK, query & bot answer.
- **Admin_User Entity:** RBAC authentication for updating college FAQs & detention data.

Verified in 3NF Normalization: Zero transitive dependencies & lossless join preservation.

Key / Legend:

- Entity (Table)
- Attribute (Column) — id = Primary Key
- Relationship
- Link / Cardinality (1:N)

Figure 2.5: Component Diagram Illustrating Multi-Tier Modular Wiring, Interface Sockets & Persistence Storage.

ASSIGNMENT 3

TOPIC: SOFTWARE PROJECT LIFE CYCLE • SDLC MODELS & COMPARATIVE ANALYSIS

1. What is Project Life Cycle and its Main Characteristics?

Definition: A **Project Life Cycle** represents the structured sequence of phases that a software development project traverses from its initial conception and requirements elicitation to its final deployment, operational maintenance, and eventual retirement. In software engineering, it establishes an organized managerial framework to ensure software quality, on-time delivery within budgetary constraints, and verifiable alignment with institutional objectives.

Standard Phases of a Software Project Life Cycle:

1. *Requirement Analysis & Specification:* Capturing functional requirements (14 FAQ domains, student login, voice interaction) and non-functional requirements (security, sub-second latency, responsive UI).
2. *System & Architectural Design:* Formulating UML blueprints (Use Case, Class, Sequence), database normalization (3NF), and REST API route contracts.
3. *Implementation & Coding:* Translating designs into executable source code using Python (Flask), SQLite3, and modern frontend web standards.
4. *Integration & Testing:* Executing unit tests (test_suite.py), functional validation, and cross-browser quality checks.
5. *Deployment & Cloud Delivery:* Releasing the application into the live production cloud environment on Render with continuous delivery hooks.
6. *Operations & Maintenance:* Monitoring system uptime, updating FAQ knowledge bases, and patching dependencies.

Main Characteristics of a Project Life Cycle:

- **Sequential & Phased Progression:** Work is partitioned into well-defined phases where each phase possesses formal entry criteria, distinct activities, and verified exit criteria.
- **Verifiable Milestones & Deliverables:** Each phase concludes with concrete, inspectable deliverables (e.g., SRS document, UML architectural dossier, source code repository, test execution matrix).
- **Inverse Risk-to-Cost Progression:** Project risk and uncertainty are highest at the project's inception and decrease progressively as requirements solidify; conversely, the financial and temporal cost of accommodating requirement changes increases exponentially in later phases.
- **Stakeholder Governance & Traceability:** Facilitates structured oversight by mentors, academic guides, and external examiners through scheduled milestone reviews.

2. Explain following SDLC models.

i) Waterfall Model (Classical Linear Sequential Model):

The Waterfall Model is a rigid, linear software development methodology where each phase (Requirements → System Design → Implementation → Testing → Deployment → Maintenance) flows strictly downward like a waterfall. A phase cannot begin until the preceding phase is 100% complete, reviewed, and signed off.

- *Strengths:* Highly structured, simple to understand and manage; clear documentation produced at each stage.
- *Weaknesses:* Extreme inflexibility to changing requirements; working software is delivered only at the very end of the life cycle; high risk of late-stage project failure.
- *Applicability:* Suitable only for projects with frozen, crystal-clear requirements and stable technology stacks.

ii) Iterative Model (Applied Methodology in College AI Chatbot):

The Iterative Model develops a software system through repeated cycles (iterations), progressively expanding functionality with each release. Rather than attempting a full implementation at once, an initial simplified prototype is developed, tested, and subsequently refined based on evaluation feedback.

- *Execution in This Project:* Our project strictly followed 6 iterative release cycles:
  - *Iteration 1:* Basic text query parsing and dictionary matching in Python.
  - *Iteration 2:* Relational database schema creation (SQLite3) and Flask REST API endpoints.
  - *Iteration 3:* Student authentication module and personalized attendance percentage/detention check.
  - *Iteration 4:* Admin control center, salted password hashing, dynamic FAQ CRUD, and CSV audit export.
  - *Iteration 5:* Web Speech API voice recognition (STT), speech synthesis (TTS), Light/Dark themes, and PWA setup.
  - *Iteration 6:* Automated unit test suite execution (test_suite.py), documentation, and Render cloud deployment.
- *Strengths:* Early working software visibility; early detection of architectural flaws; easily accommodates user feedback.

iii) Spiral Model (Risk-Driven Evolutionary SDLC):

Proposed by Barry Boehm, the Spiral Model combines the systematic, phased aspects of the Waterfall model with the iterative prototyping of evolutionary models, governed by intensive risk assessment. The process is visualized as a spiral with multiple loops, where each 360-degree loop traverses four distinct quadrants:

1. *Objective Setting:* Identifying specific phase goals, alternative architectural approaches, and operating constraints.
2. *Risk Assessment & Resolution:* Detailed technical analysis to identify, evaluate, and resolve risks (e.g., prototyping high-risk algorithms, benchmarking server loads).
3. *Development & Validation:* Engineering, coding, and testing the software artifact for that cycle.
4. *Planning Next Phase:* Reviewing current deliverables with stakeholders and planning the next spiral cycle.

• *Strengths:* Superior risk management; ideal for mission-critical, large-scale, high-uncertainty industrial projects.
• *Weaknesses:* High project management complexity; requires specialized risk analysis expertise; expensive for small projects.

iv) V-Shaped Model (Verification and Validation Model):

The V-Shaped Model is an advanced extension of the Waterfall model where each development (verification) phase has a corresponding testing (validation) phase mapped opposite to it, forming a symmetrical V shape. Testing plans and test cases are authored concurrently alongside the respective design documents before coding begins:

• *Requirements Analysis* maps to • *User Acceptance Testing (UAT)*.
• *System Architectural Design* maps to • *System Testing*.
• *Module & Database Design* maps to • *Integration Testing*.
• *Detailed Component Coding (Vertex of V)* maps to • *Unit Testing*.
• *Strengths:* High defect prevention rate; test cases developed early prevent requirement ambiguities; strict quality rigor.
• *Weaknesses:* Inflexible to scope alterations once development commences; does not provide early prototypes.

v) Agile Model (Adaptive Iterative Methodology):

The Agile Model is an iterative, incremental development philosophy grounded in the Agile Manifesto. It delivers software in small, rapid, time-boxed increments called **sprints** (typically 1 to 4 weeks long). At the conclusion of every sprint, a potentially shippable, fully tested product increment is evaluated by stakeholders.

• *Core Values:* Individuals and interactions over processes and tools; working software over comprehensive documentation; customer collaboration over contract negotiation; responding to change over following a plan.
• *Popular Frameworks:* Scrum (roles: Scrum Master, Product Owner, Dev Team; events: Sprint Planning, Daily Standup, Sprint Review/Retrospective) and Kanban.
• *Strengths:* Maximum flexibility; immediate response to changing client demands; continuous stakeholder collaboration and early ROI.

Comprehensive SDLC Comparison Matrix

SDLC Model	Structural Flow	Requirement Flexibility	Risk Management	Working Software Release	Best Suited For
Waterfall	Linear / Sequential	Very Low (Rigid)	Poor (Late Stage)	End of Life Cycle	Fixed, frozen, well-understood specs
Iterative	Repeated Cycles	Medium to High	Moderate	Progressive Iterations	Academic & commercial web apps (Chatbot)
Spiral	Evolutionary Loops	High	Excellent (Dedicated Quadrant)	After each risk loop	Complex, high-risk enterprise systems
V-Shaped	Symmetrical V (V&V)	Low	Moderate (Defect Prevention)	Late (Post validation)	Safety-critical systems (medical, defense)
Agile	Time-boxed Sprints	Extreme (Embraces Change)	Continuous Review	At end of each 1-4 wk sprint	Fast-evolving commercial startup software

ASSIGNMENT 4

TOPIC: SOFTWARE TESTING, QUALITY ASSURANCE, METRICS & CONFIGURATION MANAGEMENT

1] Define Software Testing?

Software Testing is the systematic engineering process of executing a software application under controlled conditions with the deliberate intent of identifying defects, errors, or gaps, and verifying that the delivered system conforms to all specified functional and non-functional requirements. Testing validates system behavior against expected outputs, measures performance benchmarks, verifies security defenses, and establishes high confidence in system reliability before production release. In the College AI Chatbot project, testing verified that all 14 FAQ domains returned accurate responses, fuzzy typo algorithms scored correctly, and authenticated sessions remained protected.

2] What is Risk Identification?

Risk Identification is the proactive, continuous project management process of recognizing, categorizing, and documenting potential technical, operational, schedule, or financial vulnerabilities before they materialize into active failures. In our project lifecycle, key identified risks included:

• *Hosting Latency Risk:* Cloud instances sleeping on Render's free tier causing ~30s response delays (mitigated by documenting cold-start behavior and adding responsive loading indicators).

• *Outdated Information Risk:* Stale FAQ answers due to institutional circular revisions (mitigated by engineering an admin portal with dynamic CRUD interfaces).

• *Typographical Ambiguity Risk:* Unusual query phrasing causing matching failures (mitigated by deploying Python difflib similarity scoring).

3] What is SCM?

Software Configuration Management (SCM) is an umbrella software engineering discipline that manages and tracks the evolution of software deliverables throughout the project lifecycle. SCM ensures product baseline integrity by governing four key pillars: Configuration Identification (labeling code, schemas, and documents), Version Control (recording chronological diffs), Change Control (evaluating and approving modifications), and Configuration Auditing. In our project, SCM was enforced using **Git** for version control, **GitHub** for remote repository tracking, and automated deployment hooks for Render.

4] Define Debugging.

Debugging is the diagnostic and corrective engineering activity conducted by developers to locate, analyze, trace, and rectify the underlying root cause of an identified software bug or runtime failure. Unlike testing (which merely demonstrates the existence of an error), debugging investigates internal program states, stack traces, variable values, and control flows to eliminate the flaw. For example, during chatbot development, debugging resolved a SQL datatype mismatch where attendance percentages stored as text strings caused numerical comparison errors in the detention engine.

5] Explain Configuration Audit.

A **Configuration Audit** is an independent formal assessment conducted to verify that a software configuration item conforms to its technical documentation and satisfies all baseline requirements:

• **Functional Configuration Audit (FCA):** Evaluates whether the developed software has successfully satisfied all functional, performance, and operational capabilities specified in the Software Requirements Specification (SRS).

• **Physical Configuration Audit (PCA):** Verifies that the physical build items — source code files, database DDL schemas, configuration manifests, test suites, and user documentation — are complete, correctly labeled, and properly archived in the repository.

6] Differentiate Between Black Box Testing And White Box Testing?

Criteria	Black Box Testing	White Box Testing
Core Focus	Verifies external functional behavior against requirements without knowledge of internal code.	Examines internal logic, source code structure, conditional branches, loops, and data paths.
Code Visibility	Code is treated as an opaque “black box”; inner implementation is completely hidden.	Full structural visibility into source code, algorithms, and database queries.
Performed By	Independent QA testers, end-users, and academic evaluators.	Software developers, backend architects, and unit test engineers.
Techniques	Equivalence Partitioning, Boundary Value Analysis, Decision Table Testing, State Transition.	Statement Coverage, Branch/Decision Coverage, Path Coverage, Condition Testing.
Project Example	Entering “When will Wagholi admissions close?” and verifying that the correct admission deadline appears in the chat UI.	Executing test_suite.py to test internal loop scoring in match_query() and PBKDF2 salt validation in check_password_hash().

7] What Do You Mean By Metrics?

Software Metrics are standardized, quantitative measurements used in software engineering to assess, track, and benchmark the size, quality, performance, complexity, and productivity of software products and development processes:

• *Product Metrics:* Lines of Source Code (SLOC), Cyclomatic Complexity, Database Table Count (4 normalized tables), and API Response Latency (< 250 milliseconds).

• *Process Metrics:* Defect density, test execution pass rate (100% pass on test_suite.py), and build failure frequency.

• *Chatbot Operational KPI Metrics:* Our admin command center displays live metrics: **Total Active FAQs**, **Total Queries Answered**, **Active Students**, and **Satisfaction Percentage** calculated from thumbs up/down user feedback.

8] What Do You Mean By Version Control?

Version Control (also known as Source Code Management or Revision Control) is an automated system that records changes to a set of files over time, enabling developers to recall specific historical revisions, compare chronological diffs, branch experimental features, and merge concurrent contributions without overwriting code. Our project utilized **Git** for local branch management and **GitHub** as the cloud repository. Git commit hashes provide complete cryptographic traceability for every bugfix, while Render's CI/CD pipeline listens to GitHub push events to deploy verified code to production automatically.

9] Explain Object Oriented Software Engineering?

Object-Oriented Software Engineering (OOSE) is a comprehensive software engineering paradigm that models a software system as a collaborative community of interacting “objects” that encapsulate both state (attributes) and behavior (methods), mirroring real-world domain entities. OOSE builds upon core principles:

• **Abstraction:** Exposing essential features while suppressing unnecessary low-level implementation details.

• **Encapsulation:** Bundling data and methods into isolated classes (e.g., Student class guarding roll number and attendance fields).

• **Inheritance:** Reusing common attributes through hierarchical generalization (e.g., Student and Admin inheriting from User).

• **Polymorphism:** Enabling different classes to respond to standard method calls in class-specific ways.

OOSE was applied in our project by translating UML Class and Use Case specifications directly into modular Flask blueprints and normalized relational database tables.

10] Explain Various Types of Testing in details.

1. **Unit Testing:** Testing the smallest testable units of code (individual functions or classes) in strict isolation. Application: Testing the difflib similarity scoring logic inside match_query() with synthetic inputs.
2. **Integration Testing:** Verifying that distinct software modules interface and exchange data correctly when combined. Application: Validating that the Flask /chat route correctly queries SQLite3, inserts history records, and returns compliant JSON.
3. **System Testing:** Evaluating the completely integrated software application as a unified system to verify end-to-end specifications. Application: Testing the complete web interface on Render across desktop and mobile devices.
4. **Acceptance Testing (UAT):** Validating that the system satisfies business goals and user expectations. Application: Demonstrating to project guide **Asst. Prof. Sheela Satav** that students can log in, check attendance percentages, and receive voice alerts.
5. **Regression Testing:** Re-executing existing test suites whenever source code is altered to confirm that bugfixes have not introduced secondary defects. Application: Re-testing FAQ matching after implementing the Dark/Light UI theme toggle.
6. **Performance & Load Testing:** Benchmarking response times and throughput under peak concurrent user loads. Application: Verifying sub-250ms API query latency during simulated simultaneous student queries.
7. **Security Testing:** Inspecting software for vulnerabilities. Application: Testing for SQL injection using parameterized queries, verifying salted PBKDF2 password hashes, and preventing Cross-Site Scripting (XSS).
8. **Usability & Accessibility Testing:** Assessing ease of use and accessibility. Application: Verifying Web Speech API audio clarity and mobile tap target sizes.

11] Differentiate Between automated Testing And Manual Testing.

Comparison Factor	Manual Testing	Automated Testing
Execution Method	Executed manually by a human tester interacting step-by-step with the user interface.	Executed automatically by testing scripts, frameworks, and continuous integration runners.
Execution Speed	Slow, laborious, and limited by human typing and visual review speeds.	Extremely fast; hundreds of test assertions execute in fractions of a second.
Human Resource & Fatigue	Prone to human error, boredom, and missed defects during repetitive regression cycles.	100% consistent, tire-free, and repeatable with identical precision every execution.
Initial Cost vs. ROI	Low initial setup cost; however, ongoing labor costs remain high over time.	Higher initial development cost to write test scripts; exceptionally high long-term ROI.
Best Suited For	Exploratory testing, UI/UX aesthetics, usability evaluations, and ad-hoc visual checks.	Regression testing, unit test suites, load/performance testing, and CI/CD validation.
Application in College AI Chatbot	Used to verify speech synthesis clarity, microphone voice recognition in ambient campus environments, dark mode visual contrast, mobile touch responsiveness, and admin dashboard usability across Chrome, Safari, and Edge.	Implemented in test_suite.py using Python's unittest framework. Automatically asserts: • Correct FAQ response matching for all 14 categories. • Typo-tolerant threshold validation (“hostal” → Hostel). • Student attendance lookup and 4-tier detention status math. • Admin password hashing cryptographic verification (100% pass rate).

Quality Assurance & Testing Conclusion

By deploying a hybrid testing strategy combining automated unit test suites with rigorous manual cross-browser usability evaluations, the **College AI Chatbot** achieved a verified defect-free baseline. The system delivers sub-250ms query latency, handles spelling variations with fuzzy scoring, protects administrative routes with salted cryptographic hashes, and ensures seamless academic self-service for all students of **G H Raisoni International Skill Tech University, Pune**.